UNIVERSIDAD DE CARABOBO

FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR

Práctica de Laboratorio 3

Entradas y Salidas Mapeadas en Memoria (MMIO) y Programación en Ensamblador MIPS32

Estudiante:
Gabriel Vargas
C.I.: 32.035.567

Profesor:
Prof. José Canache

Valencia, Octubre 2026

1. Respuestas al Cuestionario del Informe

Pregunta 1

1. Explique cómo se organiza la memoria cuando un sistema utiliza *memory-mapped I/O*. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones `lw` y `sw`?

- **Organización de la memoria en MMIO:** En un sistema con *Memory-Mapped I/O* (E/S mapeada en memoria), no existe una división física entre el espacio de direcciones de la memoria principal (RAM) y los periféricos de entrada/salida. El mapa de direcciones unificado de 32 bits ($2^{32} = 4$ GB) asigna rangos específicos de direcciones a los registros internos (control, estado y datos) de los dispositivos de hardware.
- **Región de memoria habitual:**
  - *En hardware MIPS32 real:* Se mapean en la región superior conocida como `kseg1` (que va desde la dirección `0xA0000000` hasta `0xFFFFFFFF`). Esta región es no traducida por la MMU y **no utiliza memoria caché** (*uncached*), asegurando que cada operación acceda físicamente al bus del periférico. Un estándar común para registros MMIO básicos en MIPS32 es la región a partir de `0xFFFF0000`.
  - *En el simulador MARS:* Se emula asignando variables en el segmento de datos (`.data`, que inicia convencionalmente en `0x10010000`).
- **Implicaciones para las instrucciones `lw` y `sw`:**
  1. **Sin instrucciones especiales:** No se requieren instrucciones propietarias para E/S; se emplean las mismas instrucciones de transferencia de memoria `lw` (Load Word) y `sw` (Store Word).
  2. **Lecturas (`lw`):** Ejecutar un `lw` sobre una dirección mapeada lee el estado o dato actual capturado por el sensor/periférico en lugar de consultar una celda RAM.
  3. **Escrituras (`sw`):** Ejecutar un `sw` sobre dicha dirección transmite una orden o configuración de control directamente hacia los circuitos del hardware.
  4. **Efectos secundarios (*Side Effects*):** A diferencia de la RAM, leer un registro mediante `lw` puede alterar el estado del periférico (por ejemplo, limpiar una bandera de interrupción automáticamente).

Pregunta 2

2. ¿Cuál es la principal diferencia entre *memory-mapped I/O* y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente *memory-mapped I/O*?

- **Diferencia principal:** MMIO utiliza un único espacio de direcciones compartido entre la memoria RAM y los dispositivos E/S (accedidos con el mismo bus). En cambio, la E/S por puertos (*Port-Mapped I/O* o PMIO) posee un espacio de direccionamiento

totalmente independiente y exclusivo para los periféricos, controlado mediante líneas físicas adicionales en el bus ($\overline{\text{IOR}}$ y $\overline{\text{IOW}}$).

- **Ventajas y Desventajas de cada enfoque:**

Enfoque	Ventajas	Desventajas
Memory-Mapped I/O (MMIO)	1. Simplifica la arquitectura al no requerir instrucciones especiales de E/S. 2. Permite usar todos los modos de direccionamiento de la CPU sobre los registros de E/S. 3. Facilita la protección mediante memoria virtual.	1. Consume direcciones del mapa global de memoria RAM. 2. La lógica de decodificación de direcciones debe ser más compleja (direcciones de 32 bits). 3. Requiere deshabilitar caché en la región mapeada.
Port-Mapped I/O (PMIO)	1. No reduce el espacio de direccionamiento asignable a la RAM. 2. Utiliza direcciones de puerto más cortas (ej. 16 bits), lo que simplifica la decodificación.	1. Requiere instrucciones especializadas de CPU (`IN`, `OUT`). 2. Modos de direccionamiento limitados únicamente a transferencias directas/indirectas por puerto.

- **¿Por qué MIPS32 utiliza principalmente MMIO?** MIPS32 es una arquitectura RISC basada en los principios de **ortogonalidad y simplificación del conjunto de instrucciones (ISA)**. Al optar por MMIO, los diseñadores evitan agregar opcodes y hardware dedicado para la E/S en la Unidad de Control, manteniendo la tubería (*pipeline*) limpia de 5 etapas y reutilizando la infraestructura existente del modelo de carga y almacenamiento (*Load/Store*).

Pregunta 3

3. En un sistema con *memory-mapped I/O*: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

- **Problemas derivados del solapamiento:** Si dos dispositivos comparten la misma dirección de memoria dentro del mapa de E/S, surge una **contención en el bus (*Bus Contention*)**:

  1. **Corrupción de datos:** Al intentar leer dicha dirección, ambos dispositivos intentarán colocar simultáneamente sus señales en las líneas del bus de datos, produciendo un valor lógico corrupto o indeterminado.
  2. **Daño físico al hardware:** Si un periférico intenta colocar un nivel alto (1 lógico / V_{CC}) y el otro un nivel bajo (0 lógico / GND) en la misma línea de bus, se genera un cortocircuito que provoca un sobrecalentamiento y daño permanente en los transistores de salida de los circuitos integrados.

- **Cómo se evita este conflicto:** Se evita mediante una **decodificación de direcciones estricta (*Address Decoding*)**. Un circuito lógico decodificador toma los bits superiores del bus de direcciones enviado por la CPU y activa una línea de habilitación exclusiva llamada *Chip Select* ($\overline{\text{CS}}$) o *Chip Enable* ($\overline{\text{CE}}$) asignada a un único dispositivo a la vez, manteniendo las salidas de los demás dispositivos en estado de alta impedancia (*High-Z*).

Pregunta 4

4. ¿Por qué se considera que el *memory-mapped I/O* simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

- **Simplificación del diseño del ISA:** MMIO elimina la necesidad de contar con instrucciones de máquina dedicadas exclusivamente a operaciones de entrada y salida. Todas las lecturas y escrituras hacia los periféricos se realizan utilizando las mismas instrucciones estándar de transferencia de datos (`lw` y `sw`). Esto simplifica la Unidad de Decodificación de la CPU, reduce el número de transistores necesarios en el procesador y permite realizar modificaciones directas sobre los registros del periférico mediante instrucciones aritméticas y lógicas habituales (`and`, `or`, `ori`).
- **Instrucciones adicionales requeridas en PMIO:** Si la arquitectura empleara E/S por puertos, la CPU necesitaría incorporar instrucciones de máquina especializadas en su ISA, como por ejemplo:
  - `IN $rt, puerto`: Transfiere el dato presente en un puerto de E/S hacia un registro interno de la CPU.
  - `OUT puerto, $rs`: Envía el contenido de un registro interno hacia un puerto de E/S.
  - Instrucciones de cadena por bloques como `INS` (Input String) u `OUTS` (Output String) para transferencias masivas por puerto.

Pregunta 5

5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

- **Ocurrencia a nivel de buses:**
  1. La CPU coloca la dirección objetivo (ej. `0xFFFF0000`) en el **bus de direcciones** (de 32 bits).
  2. La CPU activa las líneas correspondientes del **bus de control** indicando si es una operación de lectura ($\overline{\text{READ}}$) o escritura ($\overline{\text{WRITE}}$).

3. Durante una lectura, el periférico toma el control del **bus de datos** e impulsa los bits del registro solicitado. Durante una escritura, la CPU coloca el dato en el bus de datos y el periférico lo captura en sus pestillos internos.

- **¿Cómo sabe el hardware que debe acceder a un periférico y no a la RAM?** Esto lo determina el **Decodificador de Direcciones (*Address Decoder*)** integrado en la placa base o SoC:
  - El decodificador analiza continuamente los bits más significativos (MSB) del bus de direcciones.
  - Si los bits pertenecen al rango de memoria asignado a la RAM, el decodificador activa la señal $\overline{\text{CS}}$ de los módulos RAM.
  - Si los bits coinciden con el rango reservado para E/S, el decodificador **desactiva la RAM** y activa la línea $\overline{\text{CS}}$ del controlador del periférico correspondiente.

Pregunta 6

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

- **¿Es posible el acceso directo?** **No.** En sistemas operativos modernos con soporte multitarea y protección de memoria, los programas que se ejecutan en modo usuario (*User Mode*) tienen prohibido acceder directamente a las direcciones físicas mapeadas a los dispositivos E/S.
- **Mecanismos de protección existentes:**
  1. **Modos de Operación de la CPU:** Separación por hardware entre *Modo Usuario* (privilegios restringidos) y *Modo Kernel/Supervisor* (privilegios totales).
  2. **Memoria Virtual y MMU:** La Unidad de Gestión de Memoria (MMU) gestiona las tablas de páginas. Las páginas donde se encuentran las direcciones de MMIO se marcan con permisos exclusivos para Modo Kernel. Si un programa sin privilegios intenta acceder mediante `lw` o `sw`, la MMU genera una excepción de **Violación de Acceso / Fallo de Protección**.
  3. **Controladores de Dispositivos (*Drivers*) y `syscall`:** Para comunicarse con el hardware, el programa usuario debe solicitar el servicio al sistema operativo mediante una llamada al sistema (`syscall`). El Kernel valida los permisos del proceso y ejecuta la operación sobre el hardware de forma segura.

Pregunta 7

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

La espera activa (*Polling*) mantiene a la CPU ejecutando un bucle continuo comprobando el estado de un registro, lo cual desperdicia tiempo de procesamiento y energía. Para evitarlo se utilizan:

1. **Entrada/Salida Dirigida por Interrupciones (*Interrupt-Driven I/O*):** La CPU inicia la orden en el dispositivo y continúa ejecutando otros programas. Cuando el dispositivo finaliza su tarea o recibe datos, envía una señal física de interrupción (*IRQ*) a la CPU. La CPU pausa temporalmente la tarea actual, ejecuta la Rutina de Servicio de Interrupción (*ISR*) correspondiente y reanuda el programa.

2. **Acceso Directo a Memoria (*DMA - Direct Memory Access*):** Se utiliza un controlador hardware de DMA que se encarga de transferir bloques de datos directamente entre el periférico y la RAM sin pasar por la CPU. La CPU solo configura la transferencia al inicio y recibe una interrupción cuando el bloque se ha transferido completamente.

3. **Sincronización por Sistema Operativo:** El uso de mecanismos de desatasco como semáforos, colas de mensajes y estados de procesos (poner el hilo en estado *Bloqueado* o *En Espera* hasta que el evento de E/S despierte al proceso).

2. Análisis y Discusión de los Resultados

En la práctica de laboratorio se implementaron y probaron exitosamente dos controladores en lenguaje ensamblador MIPS32 mediante el simulador MARS, cubriendo el esquema de Entrada/Salida Mapeada en Memoria (MMIO) e interacción por espera activa (*Polling*).

2.1. Ejercicio 1: Sensor de Luminosidad

Se desarrollaron e implementaron los procedimientos `InicializarSensorLuz` y `LeerLuminosidad`.

- La inicialización envió correctamente el valor `0x1` al registro `LuzControl` usando la instrucción `sw`, entrando posteriormente en un bucle de comprobación del registro `LuzEstado`.
- La función de lectura evaluó el estado retornado: cuando el valor de estado fue `1`, leyó los datos desde `LuzDatos` retornando el valor de luminosidad en `$v0` y el código de éxito `0` en `$v1`.
- Ante una condición simulada de error (`LuzEstado = -1`), la rutina bifurcó hacia el manejador de errores devolviendo un código de estado de error `-1` en `$v1`.

2.2. Ejercicio 2: Módulo de Tensión Arterial

El procedimiento `controlador_tension` inició exitosamente la medición escribiendo el valor `1` en el registro `TensionControl`.

- El programa ejecutó un bucle de espera activa con la instrucción `beqz $t1, espera_tension` sobre el registro `TensionEstado`.
- Al cambiar el estado a `1` (medición finalizada), el programa extrajo los datos desde la memoria mapeada y los retornó simultáneamente en los registros de salida `$v0` (Sistólica) y `$v1` (Diastólica), respetando el estándar de convenciones de MIPS32.

2.3. Validación en el Simulador MARS

Dado que el simulador MARS no posee sensores físicos externos conectados al bus, la prueba en vivo de los algoritmos de *polling* se realizó mediante la modificación interactiva del segmento de datos (*Data Segment*) durante la ejecución paso a paso (*Single-Step*). Modificar el valor del registro de estado en vivo de `0` a `1` permitió validar que las rutinas rompen el bucle de espera de manera precisa y continúan la secuencia de lectura propuesta.

3. Conclusiones

El desarrollo de esta práctica de laboratorio permitió profundizar en la arquitectura de Entrada/Salida Mapeada en Memoria (MMIO) sobre procesadores MIPS32. Se comprobó experimentalmente cómo el paradigma RISC simplifica la Unidad de Control al reutilizar las instrucciones de carga y almacenamiento (`lw` y `sw`) para la interacción con periféricos. Asimismo, se comprendió la dinámica fundamental del mecanismo de espera activa (*Polling*) y la importancia de la decodificación de direcciones para evitar conflictos en el bus del sistema.